

Ω-LRD3: self-indexed column diagrams

DEFINITIONS & PROOFS / ENGLISH EDITION

Release definition, 2026-09-13. PDF · NER expander · Python · well-ordering proof.

The number 3 is a version number. Rows are finite expressions of this same system. The defining conventions are: one column is added at each seed stage, and packet indices run through b , not through $b + 1$. This document is self-contained and does not require any other Ω-LRD version.

1. Finite nested syntax

Expressions are finite, acyclic, recursively constructed column graphs. A finite graph $G = (m, E)$ has columns $0, \dots, m - 1$ and relations

$$(K, q, p, j), \quad 0 \leq q \leq p < j < m,$$

where the row K is an already constructed finite graph of this same syntax. The fields are row, root, parent, child. Empty graphs and empty columns are allowed. Every row occurrence is a proper subexpression; cyclic references and the external top as a row are forbidden. Sharing equal immutable subexpressions is only an implementation optimization.

Require root closure:

$$(K, q, p, j) \in E \implies (K, u, p, j) \in E \quad (0 \leq u \leq q).$$

The operator $\downarrow$ adds these smaller roots but never adds every smaller row. Ignore duplicate relations, retain empty columns. The graph with no columns is 0, printed $\emptyset$. The graph of a empty columns is the finite-number row a . Thus $[]$ is one, $[][]$ is two, and neither is zero.

Each bracketed list is a column. Its triple (row,parent,maximum_root) abbreviates all roots from zero through its maximum; the column position supplies the child. Merge equal (row,parent) groups by maximum root. Display finite-number rows as numbers, and other rows between braces containing their complete list. For example:

```
[ ][(1,0,0)][{ }[(1,0,0)],1,1]
```

The last relation has the two-column row $[][(1,0,0)]$, parent 1 and maximum root 1. Braces are nesting syntax, not an external ordinal operation.

2. Recursive column order

Within each column, sort maximum-root triples (K, p, q) in decreasing order of (p, K, q) . Compare sorted triples lexicographically, compare column lists lexicographically, then compare graphs from the leftmost column. At each list level a proper prefix is smaller. Row comparison recursively uses this same rule.

All recursive calls are on proper subexpressions, so comparison is finite structural recursion. The all-root and maximum-root representations give identical results. Identity is equality of the normalized graph, not equality of its expansion history or count sequence.

Adjoin an external top Limit , greater than all finite graphs. It is not a finite graph and cannot occur inside a row.

3. Finite-graph fundamental sequences

The index n is a nonnegative integer. Set $0[n] = 0$; this identity is not a strict descent step. If G is nonempty, write $x = m - 1$ and let ∂G delete its last column and all relations whose child is that column. Set $G[0] = \partial G$. If the last column has no relations, set $G[n] = \partial G$ for all n .

In the remaining case $n > 0$, choose the greatest last-column relation by (K, q, p) and write it (K, r, c, x) .

Control priority is **row, root, parent**; comparison priority is **parent, row, root**. Set

$$\ell = x - c > 0, \quad \phi_b(i) = \begin{cases} i & i < c, \\ i + b\ell & i \geq c, \end{cases} \quad E^- = \{(H, q, p, j) \in E : j < x\}.$$

Applying ϕ_b moves root, parent and child indices; it leaves the row expression unchanged. The finite generation packet is

$$P_b(K) = \begin{cases} \emptyset & K \text{ consists entirely of empty columns,} \\ \{0\} \cup \{K[t] : 0 \leq t \leq b\} & \text{otherwise.} \end{cases}$$

Here $K[t]$ is an expansion under these very rules. This is recursion on the proper subexpression K , not on the potentially large number of future expansion steps. In particular, a single expansion is a finite structural algorithm. A finite-number row, including zero, has no generation packet.

For $0 \leq b < n$, define

$$\begin{aligned} U_b &= \{(H, u, \phi_b(p), x + b\ell) : (H, q, p, x) \in E, \quad 0 \leq u \leq \phi_b(q), \\ &\quad H < K \text{ or } (H = K \text{ and } u < \phi_b(r))\}, \\ V_b &= \{(D, u, \phi_b(c), x + b\ell) : D \in P_b(K), \quad 0 \leq u \leq \phi_b(c)\}. \end{aligned}$$

Finally,

$$G[n] = \left(x + n\ell, \downarrow \left(\bigcup_{b=0}^n \phi_b(E^-) \cup \bigcup_{b=0}^{n-1} (U_b \cup V_b) \right) \right).$$

The source block $[c, x)$ appears once in place and n more times to the right. Normalization merges duplicates and closes roots. Increasing n only adds complete columns; $G[n]$ is a prefix of $G[n + 1]$, proper when the original last column has relations. The columns before the old last column are unchanged. A nonzero finite expansion is strictly smaller in recursive column order.

4. Seeds and the top

Define $A_0 = 0$ and $A_1 = []$. For $n \geq 1$, append exactly one new column:

$$A_{n+1} = A_n \parallel [(A_n, n - 1, n - 1)], \quad \text{Limit}[n] = A_n.$$

The appended triple uses the list field order $(\text{row}, \text{parent}, \text{maximum_root})$. Its row is the entire old term A_n , not the new term A_{n+1} . The maximum root $n - 1$ includes all roots from zero through $n - 1$.

Consequently A_n has exactly n columns. A_0 and A_1 have nesting depth zero; thereafter the maximum row-nesting depth is $n - 1$. For every $n \geq 0$,

$$A_{n+1}[0] = A_n.$$

The top sequence is therefore itself a column-prefix chain. Apart from the first column, the seeds have no empty columns inserted between relation columns. Ordinary expansions may still produce empty columns, which

must be retained.

5. Standard domain and theorem scope

The notation consists of `Limit` and all finite graphs reachable from it by finitely many expansions. Equivalently, its finite standard domain is the union of the descendant sets of the seeds A_n . Being parsable or satisfying the index constraints is not enough to establish standardness.

The linked paper proves the standard recursive column order, including `Limit`, well ordered in $KP_\omega + \exists$ an uncountable ordinal. Its route uses structural-depth induction and, for each finite starting graph, a fixed row pool formed from finitely many row-descendant cones, then applies the fixed-row finite-demand reflection theorem. The accompanying Lean final theorem is for this standard domain; it is an ordinary classical Lean proof rather than a weak-set-theory derivation encoded in Lean.

This statement does not extend the column well-ordering theorem to every raw finite graph. Neither a comparison of order types with Y, RPD or LRD nor a least possible axiom strength is asserted.

6. Display, count sequences and NER input

NER's default is a lossless one-line list. The additional equivalent display is a count sequence. This version deliberately has no mountain view; nested rows are not hidden by ellipses. Display labels in the unchanged script are Chinese.

For a column, fix its left prefix, make it the last column, expand at any positive index, and discard all newly appended columns. Iterate until this retained column has no relations, then delete it. The count includes that final deletion. An empty relation column has count one. Only seam zero affects the retained column, so its row packet is $\{0, K[0]\}$ for non-finite K , where $K[0]$ simply deletes the last column of K .

Counts are not a comparison key or a unique encoding of the graph. The JS implementation uses exact `BigInt` accumulation, with no signed-64-bit count ceiling. If guarded computation fails to finish, the display explicitly says so and does not present a lower bound as an exact answer.

Input	Meaning or count sequence
<code>Limit</code>	External top
<code>A0</code>	$\emptyset$
<code>A1</code>	1
<code>A2</code>	1, 2
<code>A3</code>	1, 2, 10
<code>A4</code>	1, 2, 10, 47
<code>A5</code>	1, 2, 10, 47, 240
<code>A6</code>	1, 2, 10, 47, 240, 1357

Import `Omega-LRD3.ne-rewritten.js` through NER's custom-notation control and select Ω -LRD3. Inputs include `A3[2][1]`, `Limit[3]`, natural numbers, and full column lists. `Tn` aliases `An`. Braces enclose finite row expressions, for example `[][(A2),0,0]`; `{Limit}` is rejected. `FS`, `FS_alter` and `FS_short` are identical.

The current script shares an approximately one-second checkpoint budget across calls in one event-loop task. Width, nesting, text and cache limits are implementation safeguards, not mathematical bounds or hard real-time guarantees. Resource exhaustion is not a new zero or successor case.

7. Independent Python expander

Python 3.10+ and the standard library suffice. From this directory:

```
python omega_lrd3.py 3 2 1
```

This computes `Limit[3][2][1]`. As a library:

```
from omega_lrd3 import OmegaLRD3, ZERO, TOP
a = OmegaLRD3.seed(3)
assert a == TOP[3]
assert a[0] == OmegaLRD3.seed(2)
assert OmegaLRD3.integer(2).is_integer
b = a[2]
assert b < a
```

Construct a custom graph with `OmegaLRD3(columns)`, each column containing `(row_graph, parent, maximum_root)` tuples. Rows must be finite instances of `OmegaLRD3`; `TOP` is rejected as a row. Constructors normalize root maxima and check index constraints, but do not decide standardness. All integers are exact. There are no built-in time or width limits; extremely deep syntax may hit Python's recursion limit and very large outputs may exhaust memory. The bounded release tests compare this algorithm with the independent earlier tuple reference and the unchanged JS, not with every possible input.